

The statement is **mostly correct**, but it needs a little clarification because modern Java and C++ both use multiple execution techniques.

1. C++ — Primarily a compiled language

When you write C++:

C++

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World";
    return 0;
}
```

The C++ compiler (e.g., GCC, Clang, MSVC) converts your source code **directly into machine code** for a specific platform.

C++ Source Code

↓

Compiler

↓

Machine Code / Executable

↓

CPU

For example:

Bash

```
g++ main.cpp -o main
```

This produces an executable (`main.exe` on Windows, for example) that the operating system can execute directly.

Therefore: C++ is traditionally classified as a **compiled language**.

2. Java — Compiled + interpreted/JIT-executed

Java works differently.

Suppose you write:

Java

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

First, the Java compiler (`javac`) compiles the source code into **bytecode**:

```
Java Source Code  
  ↓  
javac  
  ↓  
Bytecode (.class)  
  ↓  
JVM  
  ↓  
Machine Code  
  ↓  
CPU
```

You compile it using:

Bash

```
javac Hello.java
```

This produces:

```
Hello.class
```

The `.class` file contains **JVM bytecode**, not native machine code.

3. Where does interpretation come in?

The **JVM (Java Virtual Machine)** executes this bytecode.

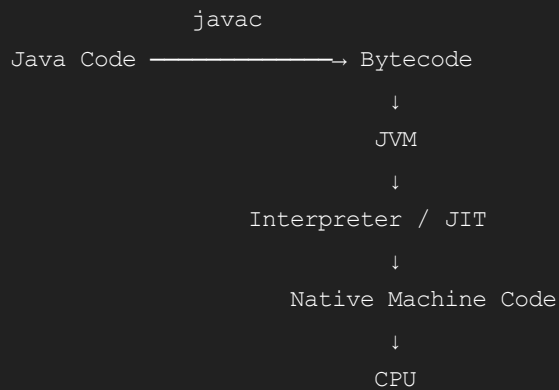
Historically, JVMs could interpret bytecode instruction-by-instruction:

```
Bytecode  
  ↓  
Interpreter  
  ↓  
Execute instruction
```

↓
Execute next instruction

Modern JVMs, such as **HotSpot**, primarily use **JIT (Just-In-Time) compilation** to compile frequently executed bytecode into native machine code at runtime.

So modern Java is better represented as:

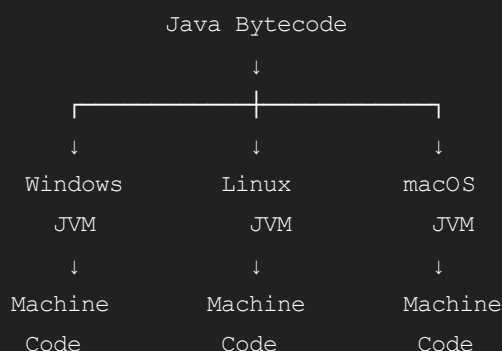


Why does Java do this?

The important benefit is **platform independence**.

A Java program is compiled into JVM bytecode rather than directly into Windows/Linux/macOS machine code.

As long as the target machine has an appropriate JVM:



That's the idea behind:

"Write Once, Run Anywhere."

C++ vs Java

Feature	C++	Java
Source code compiled?	✓ Yes	✓ Yes
Compilation output	Native machine code	JVM bytecode
JVM required?	✗ No	✓ Yes
Interpretation involved historically?	✗ No	✓ Yes
JIT compilation	Generally ✗	✓ Yes
Platform dependent executable	Generally ✓	Bytecode is platform-independent
Typical execution	Native machine code	JVM → JIT/native code

Interview-friendly answer

You can say:

C++ is traditionally a compiled language because its source code is compiled directly into platform-specific native machine code. Java is also compiled, but its source code is first compiled into platform-independent bytecode. This bytecode is executed by the JVM, which historically used interpretation and modern JVMs primarily use JIT compilation to convert frequently executed bytecode into native machine code at runtime. Therefore, Java is commonly described as a compiled and interpreted/JIT-executed language.

One important correction: **calling modern Java simply an "interpreted language" is an oversimplification.** Modern JVMs rely heavily on **JIT compilation**, so "compiled to bytecode, then executed by the JVM with interpretation and JIT compilation" is more technically accurate.

Do you like this personality?